PYTHON BRASIL 2026

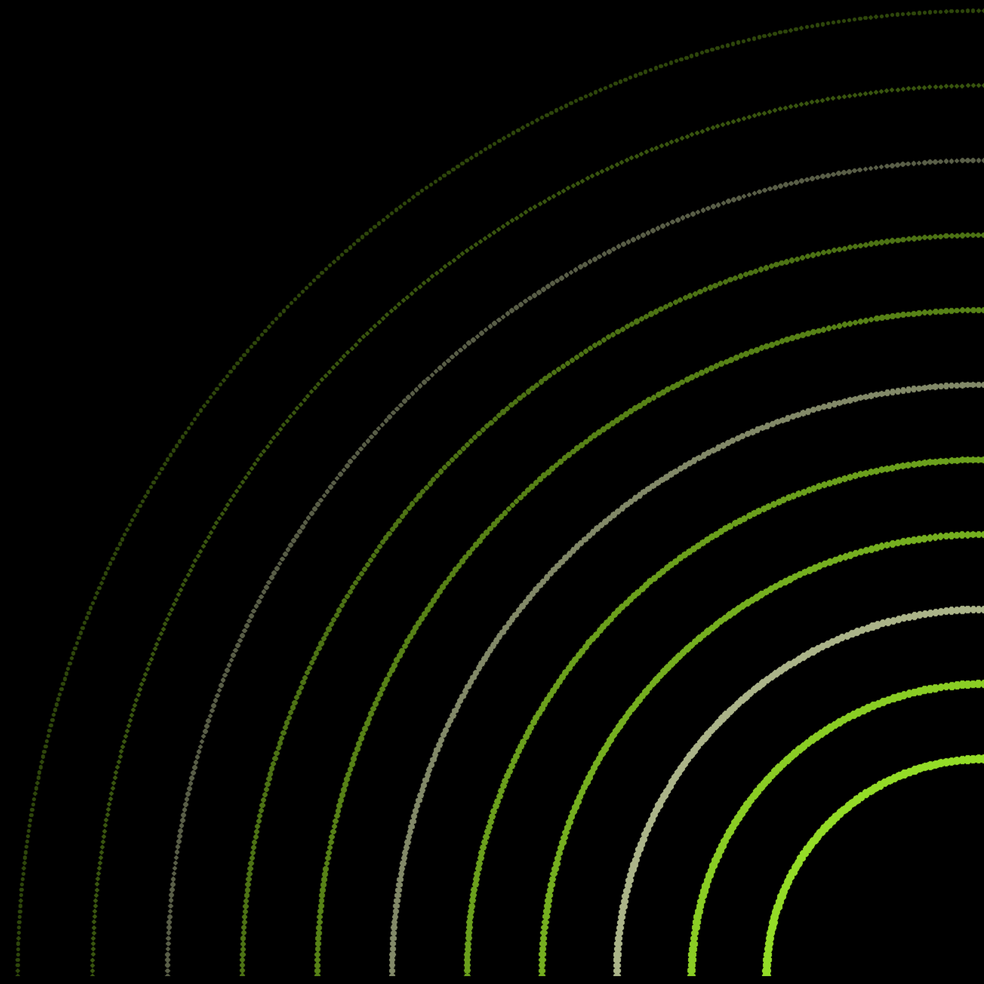
FLORIANÓPOLIS/SC · OUT 2026

ATAQUES DE SUPPLY CHAIN VIA PIP E ENTRY POINTS

O que acontece entre apertar Enter no pip install e a mão baixar.

Lucas Lemes

PYTHON BRASIL 2026



ANTES DE COMEÇAR

ESCOPO E RESPONSABILIDADE

01

EDUCACIONAL

Conteúdo voltado a defesa, detecção e threat intelligence. Nenhum artefato funcional é distribuído nesta palestra.

02

LABORATÓRIO

Todas as demos rodam offline, em Docker isolado, com segredos fictícios plantados pelo próprio lab.

03

RESPONSABILIDADE

Acesso não autorizado a sistemas é crime (Lei 12.737/2012 e 14.155/2021). Reproduza apenas em ambiente autorizado.

Pesquisa de segurança ofensiva, usada aqui para fins defensivos.

#WHOAMI



Lucas Lemes

CYBER SECURITY ANALYST

— Engenharia de software e segurança de aplicações.
— Pentest, Emulação de adversários e Red Teaming.
— Cyber Security Analyst – Banco do Brasil
— Professor FIAP - Alura

SUPPLY CHAIN | PYTHON | APPSEC | DEVSECOPS

linkedin.com/in/lucas-lemes-839359111/

ROTEIRO

AGENDA

01 O desenvolvedor como o novo perímetro

02 Anatomia do ataque: além do import

03 Vetores de propagação e engenharia social

04 Demonstração prática (live demo)

05 Estratégias de defesa e resiliência

06 Conclusão e perguntas

QUAL RISCO EXISTE NESSA AÇÃO?

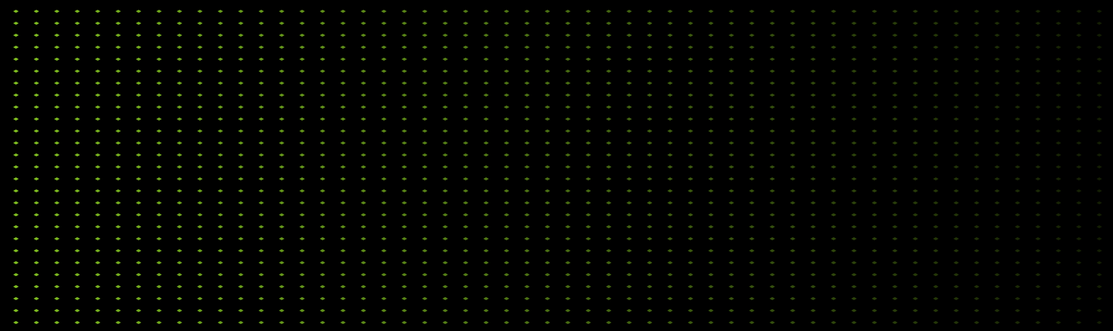


SEÇÃO 01

O DESENVOLVEDOR É O NOVO PERÍMETRO

01

Por que o alvo deixou de ser o servidor e passou a ser o seu laptop.



PARADIGMA

ONDE SE GASTA SEGURANÇA × POR ONDE ELE ENTRA

— A empresa investe em firewall, WAF, Zero Trust na borda.

— O atacante não bate na porta da frente.

— Ele manda um pacote para o laptop de quem escreve o código.

— Esse laptop guarda o que o firewall não protege.

Comprometer um engenheiro é comprometer tudo que ele acessa — de dentro.

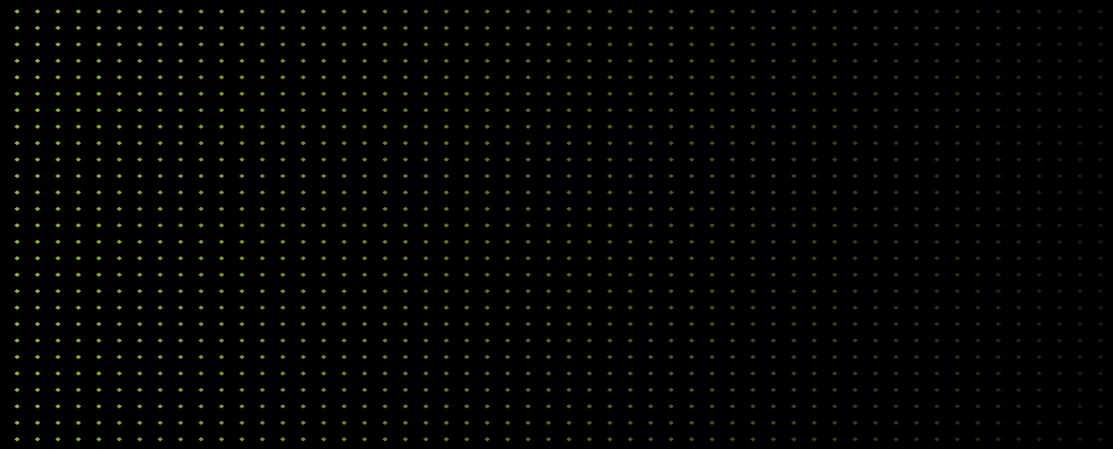
O QUE O LAPTOP GUARDA

```
.env de produção
~/.aws/credentials
token do GitHub no keychain
VPN já autenticada
kubeconfig do cluster
```

Confiança sem verificação é exatamente a superfície que o atacante explora.

O ECOSSISTEMA PYTHON CRESCEU SOB UM CONTRATO DE CONFIANÇA

O dev virou alvo de infiltração corporativa — e o vetor preferido é a cadeia de dependências.



EM NÚMEROS

ISSO NÃO É TEORIA

2 mil+

Denúncias de malware

Processadas pelo PyPI só em 2025.

119 mil

Downloads do LiteLLM

Na janela de ~2h30 com a versão envenenada no ar (mar/2026).

170+

Pacotes do Shai-Hulud

Worm que pulou de npm para PyPI roubando tokens de CI/CD.

Fontes: PyPI 2025 in review; incident reports LiteLLM/Telnyx; GitLab/Unit42/JFrog (2026).

Em vários casos o código malicioso rodou na instalação — ninguém importou nada.

ISSO NÃO É TEORIA

LiteLLM comprometido em ataque de cadeia de suprimentos

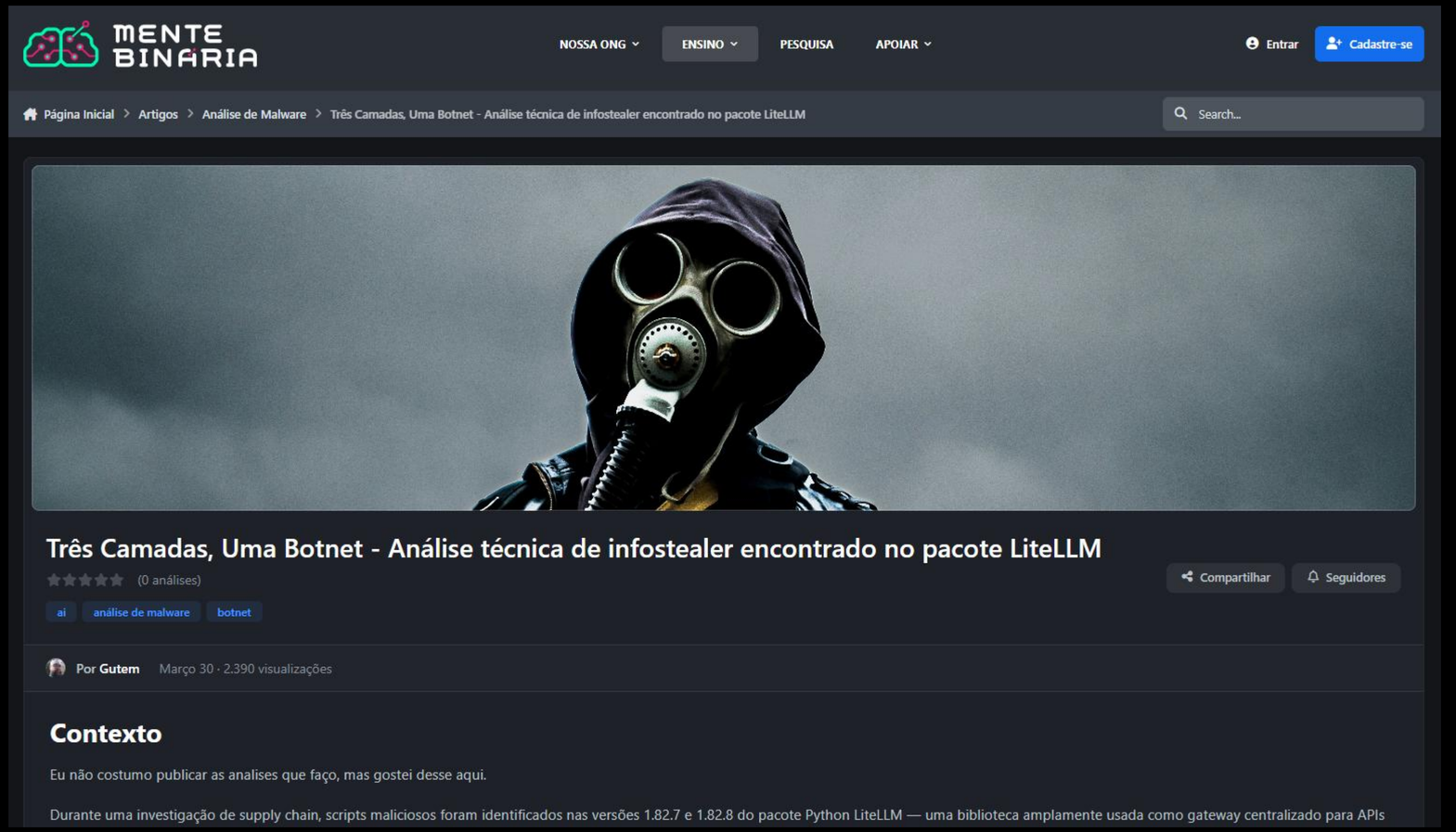
:: Da Redação :: 25/03/2026 16:58

O LiteLLM, pacote Python de código aberto amplamente utilizado por sistemas de inteligência artificial, foi **comprometido** em um ataque à cadeia de suprimentos. As versões comprometidas, identificadas como 1.82.7 e 1.82.8, foram publicadas no Python Package Index na terça-feira e baixadas inadvertidamente em ambientes de desenvolvimento e nuvem. De acordo com especialistas da Sonatype, os pacotes maliciosos ficaram disponíveis por pelo menos duas horas em 24 de março. Considerando que o pacote tem cerca de três milhões de downloads diários, os pesquisadores afirmam que os hackers podem ter alcançado um número significativo de vítimas durante esse período.

Malware extrai credenciais e instala backdoor persistente

Os pacotes manipulados continham código malicioso projetado para extrair dados sensíveis, incluindo credenciais de nuvem, chaves de API e carteiras de criptomoedas. O malware também mantinha acesso por meio da instalação de um downloader persistente, permitindo que os atacantes obtivessem acesso mais profundo e realizassem invasões subsequentes. Adam Reynolds, pesquisador sênior de segurança da Sonatype, observou que o malware apresenta comportamentos incomuns, como se comunicar com seu endpoint de comando a cada 50 minutos. Esse longo intervalo pode ajudar a evadir ambientes de sandbox que normalmente executam amostras por períodos mais curtos.

ISSO NÃO É TEORIA



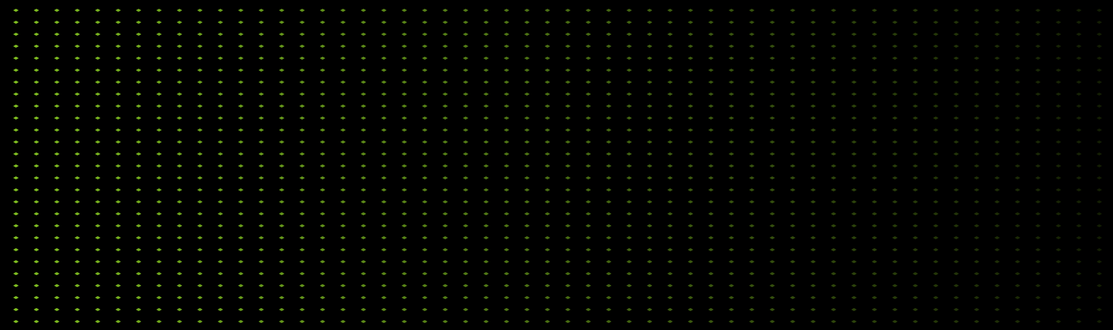
https://www.mentebinaria.com.br/artigos/an%C3%A1lise-de-malware/tr%C3%AAs-camadas-uma-botnet-an%C3%A1lise-t%C3%A9cnica-de-infostealer-encontrado-no-pacote-litellm-r120/

SEÇÃO 02

ANATOMIA DO ATAQUE: ALÉM DO IMPORT

02

Quatro momentos em que o código de uma dependência roda sem você importar.



A INTUIÇÃO ERRADA

"SÓ RODA QUANDO EU FAÇO IMPORT"

— A crença: se eu não importar, estou seguro.

— Falsa em pelo menos quatro momentos diferentes.

— O código roda antes, durante e depois do import.

— Nenhum deles é bug: são recursos legítimos do Python.

O ataque não quebra a linguagem — ele abusa da confiança na instalação.

OS QUATRO MOMENTOS

1 · Instalação – setup.py
2 · Startup do interpretador – .pth
3 · Entry points – plugins e scripts
4 · Import-time – __init__.py

MOMENTO 1 · INSTALAÇÃO

setup.py RODA NO pip install

setup.py

```
from setuptools import setup
from setuptools.command.build_py import build_py

class BuildComBrinde(build_py):
    def run(self):
        # roda no 'pip install', antes do import
        coletar_segredos()   # lê ~/.aws, .env
        super().run()        # e constrói normalmente

setup(name="docbr-utils", version="0.9.9",
      cmdclass={"build_py": BuildComBrinde})
```

O PONTO

— sdist exige build → executa o setup.py do autor.
— Código arbitrário, seu usuário, antes de qualquer import.
— pyproject.toml (PEP 517) não salva: o backend roda o setup.py igual.
— Só wheel (--only-binary) fecha ESTE vetor — mas não os próximos.

DEFESA

Hashes travados abortam antes do build rodar.

MOMENTO 2 · STARTUP DO INTERPRETADOR

O ARQUIVO QUE RODA SEM VOCÊ PEDIR

telemetria_br.pth

```
# telemetria_br.pth
# (raiz do site-packages, vem no wheel)
import _tele_hook

# _tele_hook.py
# roda em TODO python, pytest, pip...
executar_payload(gatilho="startup")
```

O PONTO

— Linha do .pth que começa com 'import' é executada.
— Roda a cada inicialização de qualquer interpretador do ambiente.
— Sem cron, sem .bashrc, sem rastro no history.
— Técnica dos worms Shai-Hulud (2026). Viaja no wheel.

DEFESA

Audite site-packages/*.pth procurando linhas 'import' ou padrões como subprocess, base64 ou exec

MOMENTO 3 · ENTRY POINTS

O RECURSO LEGÍTIMO QUE VIRA ARMA

pyproject.toml

```
[project.scripts]
br-deploy = "dev_helper.cli:main"
# vira um comando no PATH do venv

[project.entry-points.pytest11]
plugin = "dev_helper.pytest_plugin"
# o pytest carrega sozinho
```

O PONTO

— console_scripts: um binário novo no PATH do venv.
— Nome familiar (br-deploy) → você digita e roda o do atacante.
— Grupos de plugin (pytest11) carregam automaticamente.
— Basta estar instalado: o próximo pytest no CI já executa.

DEFESA

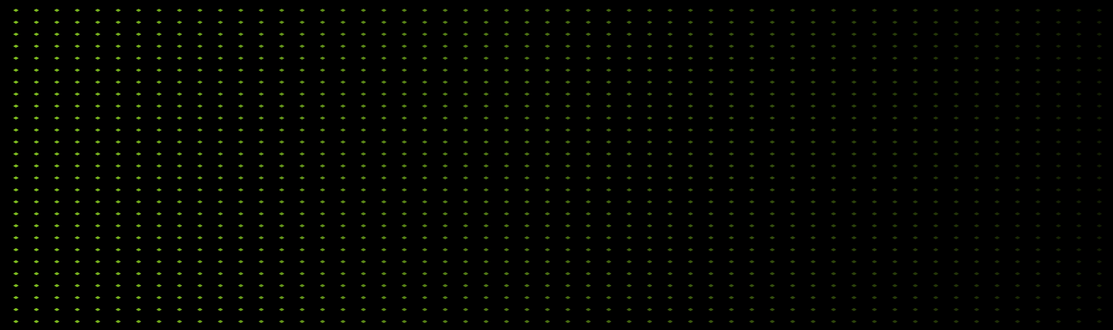
Um venv por projeto isola scripts e entry points.

SEÇÃO 03

VETORES DE PROPAGAÇÃO E ENGENHARIA SOCIAL

03

Como o pacote malicioso chega até o seu pip install.



VETOR 1 · TYPOSQUATTING

UM TRAÇO ENTRE SEGURO E COMPROMETIDO

— Nomes parecidos com libs populares, à espera do erro de digitação.

— python-requests por requests; beautifulsoup por beautifulsoup4.

— O recorte Brasil: brutils, validate-docbr, pycep-correios, pycpfcnpj.

— O pacote falso funciona (valida o CPF) — a carga fica no setup.py.

Campanhas reais em 2025–2026 publicaram dezenas de typosquats em lote.

ALVOS BR TÍPICOS

```
requests          → python-requests
validate-docbr    → validate_docbr
brutils           → br-utils
pycpfcnpj         → docbr-utils
```

VETOR 2 · DEPENDENCY CONFUSION

O PIPELINE ESCOLHE A VERSÃO MAIOR

install.sh

```
# índice privado + público como fallback:
pip install \
  --index-url https://pypi.interno/simple/ \
  --extra-index-url https://pypi.org/simple/ \
  pagamentos-internos

# privado: 1.0.0   público: 9.9.9
# → instala a 9.9.9 do atacante
```

O PONTO

— Nomes internos vazam em requirements, Dockerfile, stack trace.
— Atacante publica o mesmo nome no PyPI com versão altíssima.
— O resolver escolhe a maior: a pública, a do invasor.
— Alex Birsan (2021): Apple, Microsoft, 30+ empresas, US$130k+.

DEFESA

Um índice só, que espelha o público. Nada de --extra-index-url.

VETOR 3 · COMPROMETIMENTO DE MANTENEDOR

QUANDO A FONTE CONFIÁVEL É SEQUESTRADA

LITELLM

Token de publicação vazou via dependência de CI (Trivy). Coletor de credenciais rodando na instalação: 2h30 no ar, 119 mil downloads.

TELNYX

Pacote legítimo e popular; versão maliciosa publicada. Mais de 740 mil downloads no mês do incidente (mar/2026).

SHAI-HULUD

Worm que rouba tokens de publicação (PyPI/npm) e os usa para publicar mais pacotes. Credencial roubada vira munição.

Não depende de erro do dev: a conta de um mantenedor legítimo é comprometida.

E COM IA?

SUPPLY CHAIN & IA

A IA entra nos dois lados: como **alvo**, como **vítima do mesmo vetor** e como **cúmplice involuntária** — e em todos o código roda no pip install.

01 ALVO

LITELLM

O gateway que o time usa para falar com LLMs recebeu versões maliciosas que roubavam credenciais **na instalação**.

Mar-2025

02 VÍTIMA DO VETOR

PyTorch / torchtriton

Pacote de mesmo nome no PyPI público venceu o oficial e exfiltrou SSH keys e variáveis das máquinas de treino (GPU + nuvem).

Dez-2022

03 CÚMPLICE

Slopsquatting

LLMs de código alucinam nomes: 19,7% não existem. O atacante registra e espera.
PoC: huggingface-cli → 30 mil downloads.

2025

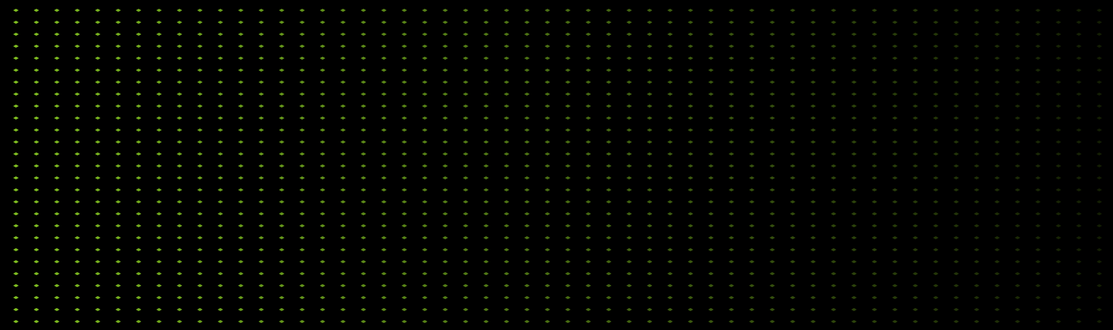
A IA é alvo (LiteLLM), vítima do vetor clássico (PyTorch) e agora cúmplice (slopsquatting). A superfície continua a mesma: a confiança no ato de instalar.

SEÇÃO 04

DEMONSTRAÇÃO PRÁTICA (LIVE DEMO)

04

Tudo offline, em Docker isolado, com segredos fictícios. Olhem o terminal do coletor.



LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 1 · typosquat + setup.py: um traço e os segredos vazam no build.

— Demo 2 · .pth: um install, e todo interpretador passa a chamar de volta.

— Demo 3 · entry points: br-deploy no PATH e plugin carregado pelo pytest.

— Demo 4 · confusion: pede o interno, recebe a 9.9.9 pública.

Zero import malicioso escrito pelo dev. Dois terminais: coletor × vítima.

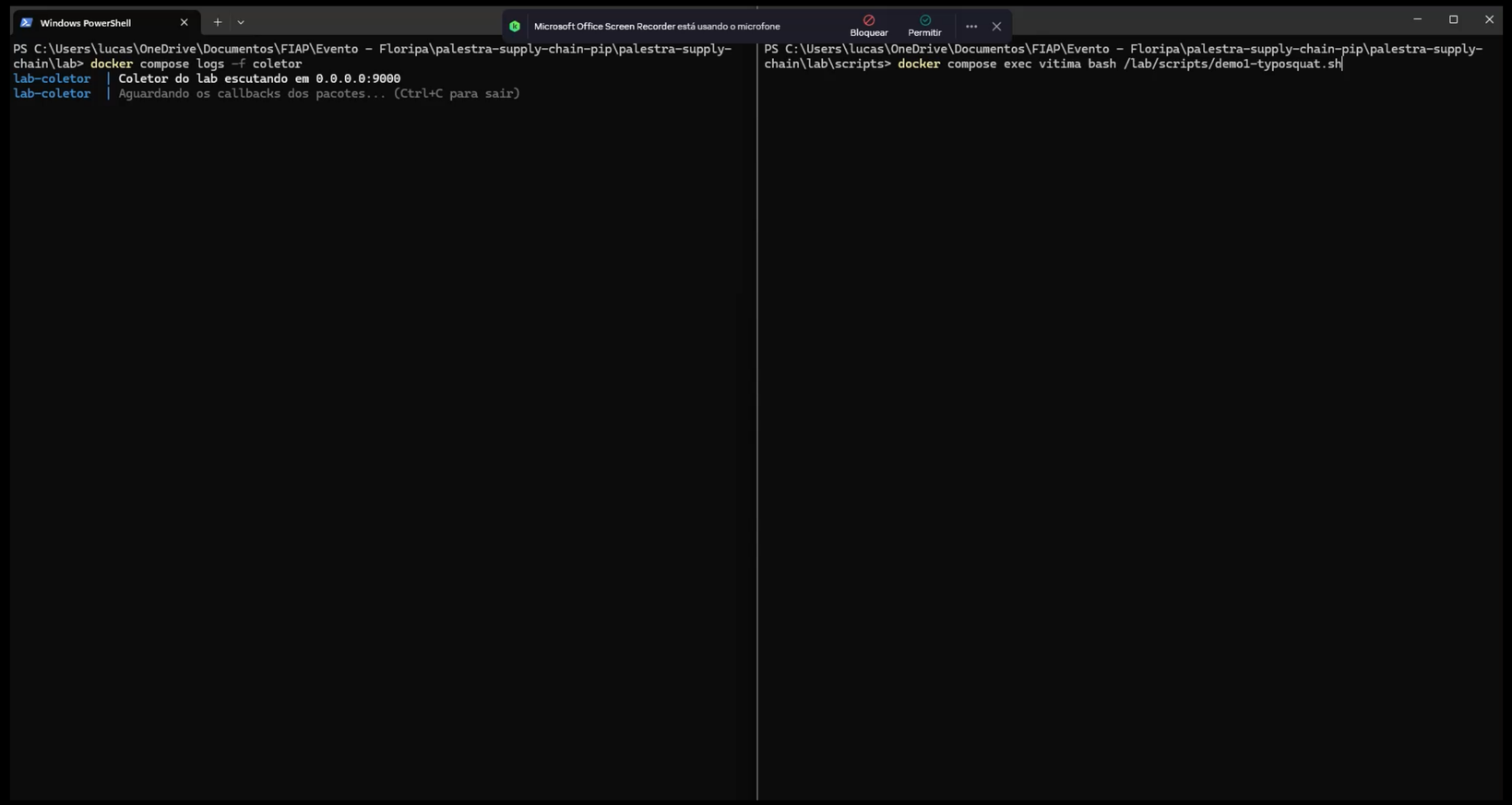
PLACAR

```
4 instalações
4 callbacks no coletor
0 imports maliciosos
```

LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 1 · typosquat + setup.py: um traço e os segredos vazam no build.



LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 1 · typosquat + setup.py: um traço e os segredos vazam no build.

```python
# setup.py — o atacante sobrescreve a etapa de build
from setuptools.command.build_py import build_py

class BuildComBrinde(build_py):
    def run(self):
        exfiltrar_segredos()  # roda no pip install
        super().run()       # e ainda constrói normal

setup(name="docbr-utils", cmdclass={"build_py": BuildComBrinde})
```

O PONTO

O dev quer docbrutils, mas digita docbr-utils. O nome com hífen estava livre.

$ pip install docbr-utils ← nenhum import. só o install.

LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 2 · .pth: um install, e todo interpretador passa a chamar de volta.

LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 2 · .pth: um install, e todo interpretador passa a chamar de volta.

```
# telemetria_br.pth (vai na raiz do site-packages)

import _tele_Hook

# o Python executa TODA linha .pth que começa com 'import'
# a cada inicialização do interpretador — sem ninguém importar.

# _tele_hook.py

executar_payload(gatilho="startup do interpretador")
```

O PONTO

Um wheel instala um arquivo .pth na raiz do site-packages. A técnica dos worms Shai-Hulud (2026).

$ python -c "print('hello')" ← dispara o payload

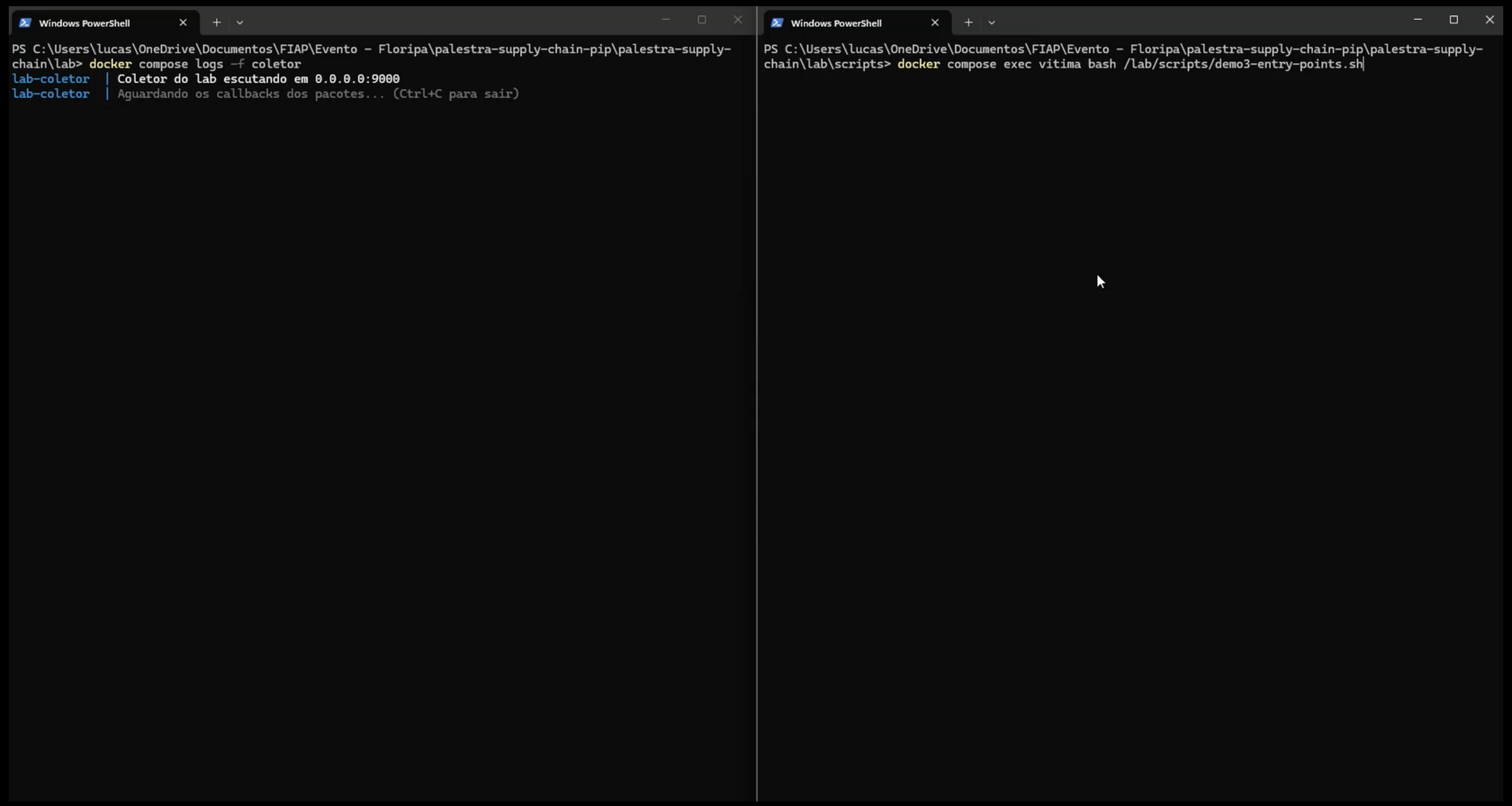
Roda em todo start de Python: python, pytest, pip, jupyter.

O ponto-chave: o dev acredita que só import é perigoso. Aqui o pacote nunca é importado — e mesmo assim executa.

LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 3 · entry points: br-deploy no PATH e plugin carregado pelo pytest.



LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 3 · entry points: br-deploy no PATH e plugin carregado pelo pytest.

```toml
# (A) vira um comando no PATH do venv
[project.scripts]
br-deploy = "dev_helper.cli:main”

# (B) o pytest carrega este plugin SOZINHO ao iniciar
[project.entry-points.pytest11]
dev_helper = "dev_helper.pytest_plugin"
```

O PONTO

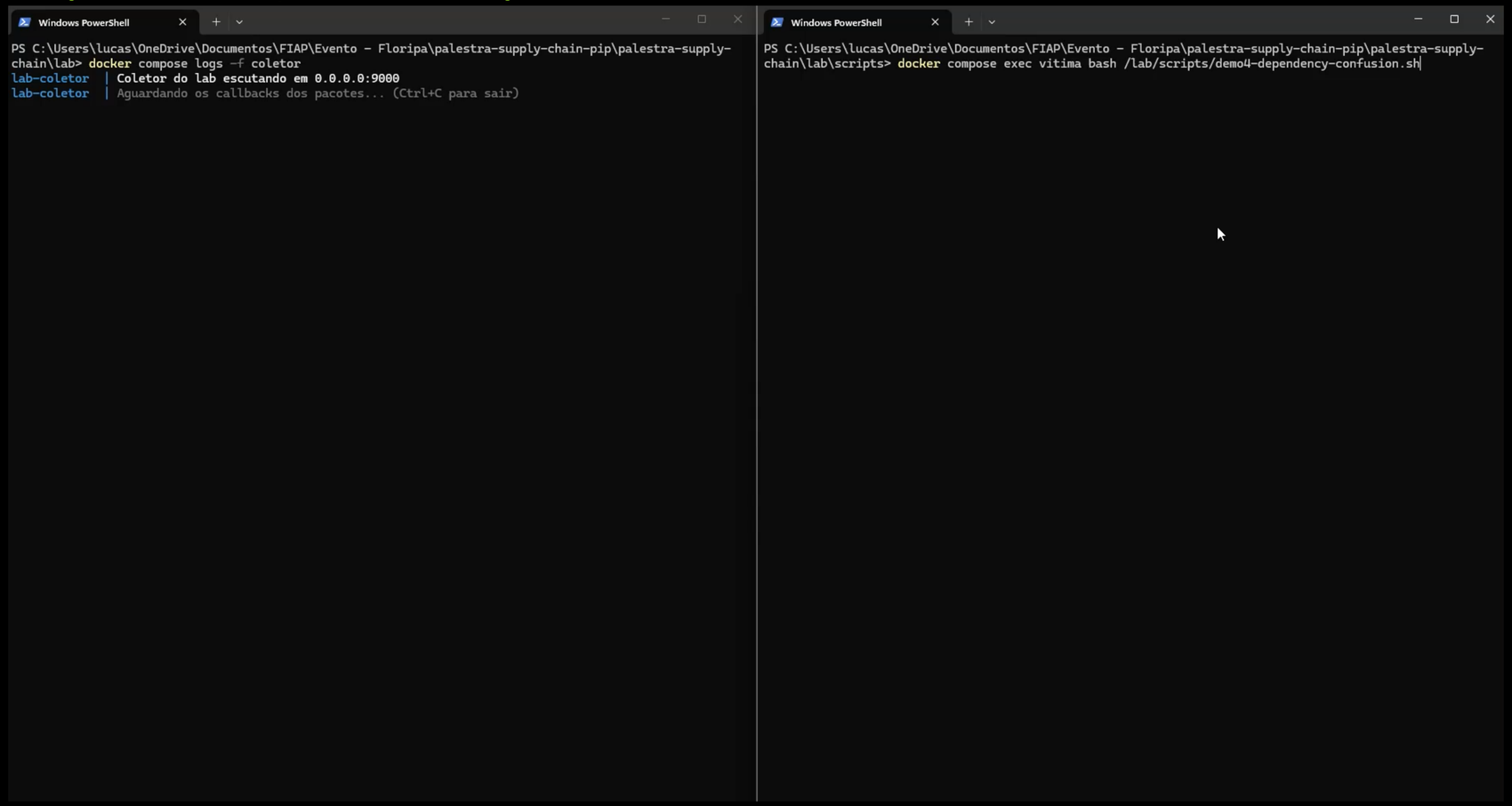
Recurso legítimo do empacotamento, abusado de dois jeitos num pyproject.toml.

$ br-deploy · $ pytest ← os dois executam o atacante

LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 4 · confusion: pede o interno, recebe a 9.9.9 pública.



LIVE DEMO

QUATRO INSTALAÇÕES, QUATRO COMPROMETIMENTOS

— Demo 4 · confusion: pede o interno, recebe a 9.9.9 pública.

```
# a empresa deixa o público como fallback

$ pip install --index-url PRIVADO \
    --extra-index-url PUBLICO pagamentos-internos

# pip vê 1.0.0 e 9.9.9 → escolhe a MAIOR → a do atacante
```

O PONTO

Índice Privado = pagamentos-internos 1.0.0

Índice Público(atacante) = pagamentos-internos 9.9.9

origem instalada: PyPI-publico-do-atacante ← o dev não errou uma tecla

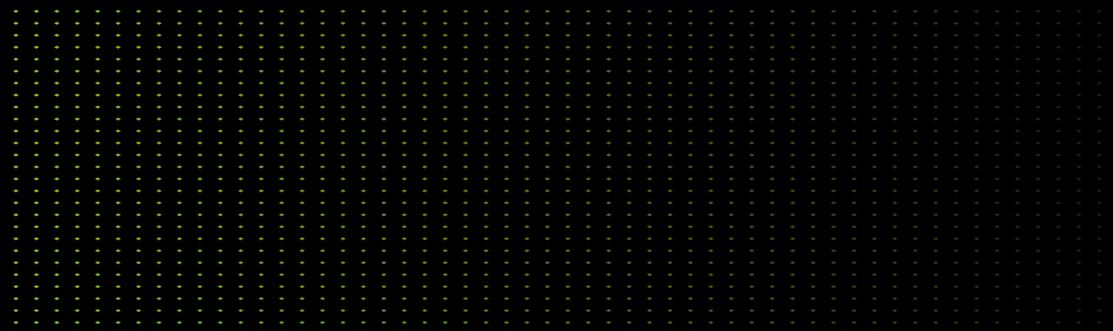
Com dois índices, o pip resolve pela versão mais alta, não pela origem. O atacante só precisa publicar um número maior.

SEÇÃO 05

ESTRATÉGIAS DE DEFESA E RESILIÊNCIA

05

Deslocar a confiança de 'eu confio no pacote' para 'eu verifico o artefato'.



DEFESA EM CAMADAS

NENHUMA BALA DE PRATA, CINCO CAMADAS

— 1 · Fixar e verificar: hashes travados + lock files (PEP 751).

— 2 · Cooldown: ignore releases recém-nascidas (--uploaded-prior-to, exclude-newer).

— 3 · Auditoria: pip-audit e uv malware-check como gate no CI.

— 4 · Matar o confusion: um índice só, que espelha o público.

— 5 · Higiene: um venv por projeto; Trusted Publishing (OIDC) ao publicar.

66% dos malwares caem em até 4h — cooldown sozinho já corta quase tudo.

NO TERMINAL

```
pip install --require-hashes \
  -r requirements.txt

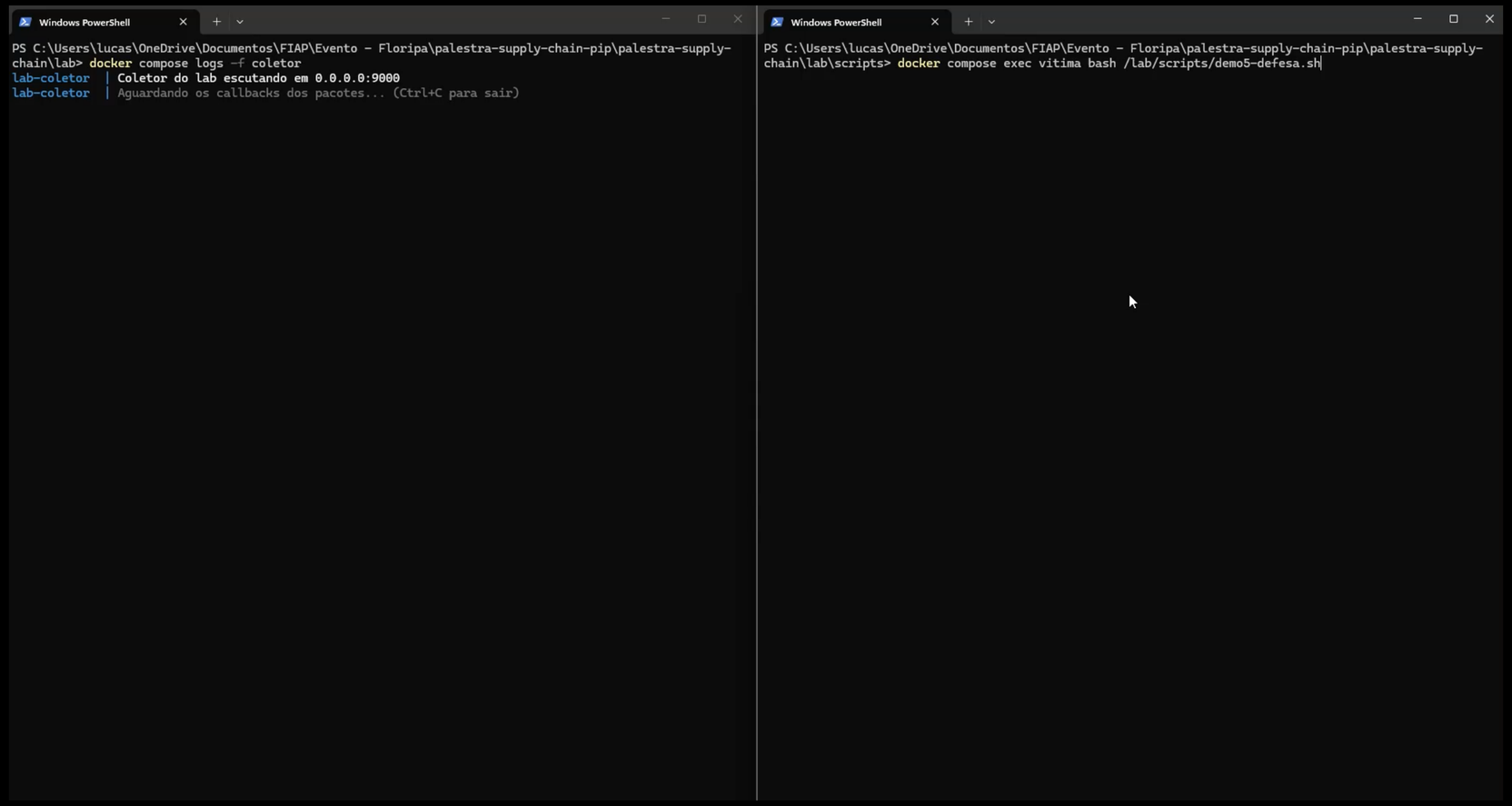
pip install \
  --uploaded-prior-to=P3D pkg

uv sync --exclude-newer "7 days"
pip-audit   # gate no CI
```

LIVE DEMO

A DEFESA

— Demo 5 · As estratégias de defesa.



LIVE DEMO

A DEFESA

— Demo 5 · As estratégias de defesa.

- → **Hashes travados (--require-hashes) barram o pacote adulterado**
- → **Cooldown de dependência ignora releases recém-nascidas**
- → **pip-audit no CI + um índice só + um venv por projeto**

Verifique antes de confiar. O seu laptop é o novo perímetro — ele guarda os segredos que o firewall não protege.

PARA LEVAR PARA CASA

TRÊS FRASES

NÃO EXISTE IMPORT PERIGOSO

O perigo está no ato de instalar e no ambiente que carrega. setup.py, .pth e entry points: antes, durante e depois do import.

O LAPTOP É O PERÍMETRO

Ele guarda os segredos que o firewall não protege. Por isso é o alvo favorito do Red Team hoje.

A DEFESA É ACESSÍVEL

Hashes, cooldown, pip-audit, um índice, um venv. Não é caro: é disciplina. Verifique antes de confiar.

Da próxima vez que digitar pip install, lembre dos dois terminais.


JORNADA
PORTFÓLIO TECH
CONSTRUA O QUE O MERCADO BUSCA
APRENDA A CRIAR UM PORTFÓLIO QUE MOSTRA
O SEU POTENCIAL E DESTAQUE SEU PERFIL
NO MERCADO DE TECNOLOGIA

FIAP

OBRIGADO!

Perguntas, críticas e ideias são bem-vindas.

E-MAIL lucasdasilvalemes89@gmail.com

LINKEDIN linkedin.com/in/lucas-lemes-839359111/

GITHUB https://github.com/LucasLemes89/ATAQUES-DE-SUPPLY-CHAIN-VIA-PIP-E-ENTRY-POINTS



